

Lecture 9

Testing 1

Outline

- Definition and objectives
- Black box testing techniques
 - Equivalence partitioning technique
 - Boundary value analysis technique
 - Decision table technique
 - State transition testing technique
 - Pairwise testing technique

References

- Daniel Galin, Software Quality Assurance: From theory to implementation, chapter 9
- ISTQB, Certified Tester Advanced Level Test Analyst (CTAL-TA) Syllabus

Definition and objectives

Software testing

- Testing is the largest consumer of SQA resources
- **Survey Data:**
 - **Budget:** ~24% of the project development budget
 - **Time:** ~27% of the project schedule
 - *Reality:* Managers often plan for more time (45%) but cut it down due to deadline pressures
- Testing requires the **direct running of code**

Software testing

- Software testing is a **formal process** carried out by a **specialized testing team** in which a software unit, several integrated units, or an entire software package are examined by **running the programs** on a computer
- **Requirements:**
 - Approved test procedures
 - Approved test cases

Key characteristics of testing

- **Formal:** Part of the project plan; ad-hoc checks by colleagues do not count
- **Specialized Team:** performed by independent teams or consultants to eliminate bias
 - *Why?* Developers often struggle to find errors in their own work
- **Running the Programs:** If the code isn't executed (e.g., code review), it is not “testing”
- **Approved Procedures & Cases:**
 - Testers follow the plan strictly
 - No omission of redundant cases or addition of new unapproved cases during the process

Software testing objectives

Direct Objectives

- To **identify and reveal** as many errors as possible.
 - *Quote:* "If your goal is to show the absence of errors you won't discover many." (Myers, 1979).
- To bring software to an **acceptable level of quality** (not necessarily "bug-free," but tolerable for the user).
- To perform tests **efficiently** within budget and schedule limits.

Indirect Objective

- To compile a record of errors for use in **error prevention** (corrective and preventive actions)

Verification, Validation, and Qualification

Concept	Definition Summary	Focus
Verification	Evaluating if the product of a phase satisfies the conditions of the previous phase	Consistency: “Are we building the product right?”
Validation	Evaluating if the system satisfies specific requirements	Customer Needs: “Are we building the right product?”
Qualification	Determining if the system is suitable for operational use	Operations: Maintenance and coding standards

Black box testing techniques

Test techniques

- **Purpose:** To systematically identify
 - **Test Conditions** (What to test)
 - **Test Cases** (How to test)
 - **Test Data** (Input values)
- Some techniques are specific to certain test levels (e.g., Unit vs. System), while others are applicable universally
- Testers rarely rely on a single method. Best results are achieved by combining multiple techniques to ensure broader coverage

Test techniques

Choice of technique depends on:

- **System Characteristics:** Complexity, Risk levels, Expected defect types
- **Requirements:** Regulatory standards, Customer contracts, SDLC model
- **Project Constraints:** Time, Budget, Available tools & Documentation
- **Team:** Tester knowledge and skills

Black box testing techniques

- **Black Box Testing** (also called behavioral or behavior-based techniques) focuses on inputs and outputs without examining the internal code structure
- **Goal:** To increase testing efficiency and improve coverage of potential error conditions
- **Key Techniques:**
 - Equivalence partitioning
 - Boundary value analysis
 - Decision table
 - State transition testing
 - Pairwise testing

Equivalence partitioning technique

- **Concept:** Dividing the input domain into partitions (or equivalence classes)
- **Assumption:** The program processes every value within a specific class identically
- Class types:
 - Valid equivalence class: A set containing only valid states (correct inputs)
 - Invalid equivalence class: A set containing only invalid states (error inputs)
- **Benefit:** Eliminates redundancy; there is no need to test every single possible value

Equivalence partitioning technique

- **For ECs:**
 - Each equivalence class should be covered by at least one test case. -> total testcases number should be more than the number of equivalence classes.

Equivalence partitioning technique

Input condition	Equivalence class
a range	one valid and two invalid classes
a specific value	one valid and two invalid classes
an element of a set	one valid and one invalid classes
boolean	one valid and one invalid classes
an array	one valid and two invalid classes
a string	similar to array of characters

Equivalence class for input conditions

Boundary value analysis technique

- **Concept:** An extension of Equivalence Partitioning
- **Principle:** Errors occur more frequently at the “boundaries” (edges) of input ranges than in the center
- **Methodology:** Instead of selecting random values, select:
 - Lower boundary value
 - Upper boundary value
 - The boundary value

Boundary value analysis technique

Condition	Test cases
input a range [a,b]	Values a and b and just above and just below a and b
input a number of values	The minimum, maximum numbers, values just above and below minimum and maximum
output a range [a,b]	Choose test case to create output values: a and b and just above and just below a and b
output a number of values	Choose test case to create output values: the minimum, maximum numbers, values just above and below minimum and maximum

Guidelines for Boundary value analysis

Decision table technique

- **Concept:** A technique used to model system logic as rules based on combinations of multiple inputs
- **Table Structure:**
 - **Conditions:** Input factors (e.g., Is Member? Has Coupon?)
 - **Actions:** Expected outputs (e.g., Apply 10% Discount, Free Shipping)
 - **Rules:** Columns representing every possible combination of True/False for the conditions

Decision table technique

When to Use Decision Tables?

- When business logic is complex (e.g., “If X and Y are true, but Z is false, then...”)
- To ensure **Completeness**: Helps verify that no logical combination has been overlooked
- Guarantees that every combination of conditions leads to a defined result

Decision table technique

Conditions	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6
Male	T	T	T	F	F	F
Female	F	F	F	T	T	T
< 25	T	F	F	T	F	F
≥ 25 & < 65	F	T	F	F	T	F
≥ 65	F	F	T	F	F	T
Actions						
Fee(\$)	3000	1000	1500	500	500	1500

Decision table for car insurance

State transition testing technique

- **Concept:** A technique used to model the behavior of a system by showing its possible states and valid state transitions
- **Key Elements:**
 - **State:** A situation or condition the system is in (e.g., “Logging In”, “Waiting”)
 - **Event:** An occurrence that initiates a transition
 - **Guard Condition:** A condition that **additionally qualifies** an event
 - **Action:** An operation performed by the software during a transition
 - **Transition:** A change from one state to another, assumed to be instantaneous

State transition testing technique

Based on the 5-step process for applying state-based testing concepts:

1. **Define Perspective:** View the system from the perspective of the user, the system itself, or a real-world object
2. **Identify States:** Understand all various states, including initial and final states
3. **Map Transitions:** Identify the events, conditions, and actions that apply (and those that don't) for each state
4. **Model the System:** Use a graph (diagram) or table to model the system and predict behavior
5. **Verify:** Verify that the correct action and next state occur for each transition

State transition testing technique

Coverage Criteria:

1. All States Coverage

- **Focus:** The coverage items are the states
- **Requirement:** Test cases must ensure that **all states** are exercised
- **Measurement:** $(\text{Exercised States} / \text{Total States}) \%$
- **Evaluation:** It can typically be achieved without exercising all transitions

State transition testing technique

Coverage Criteria:

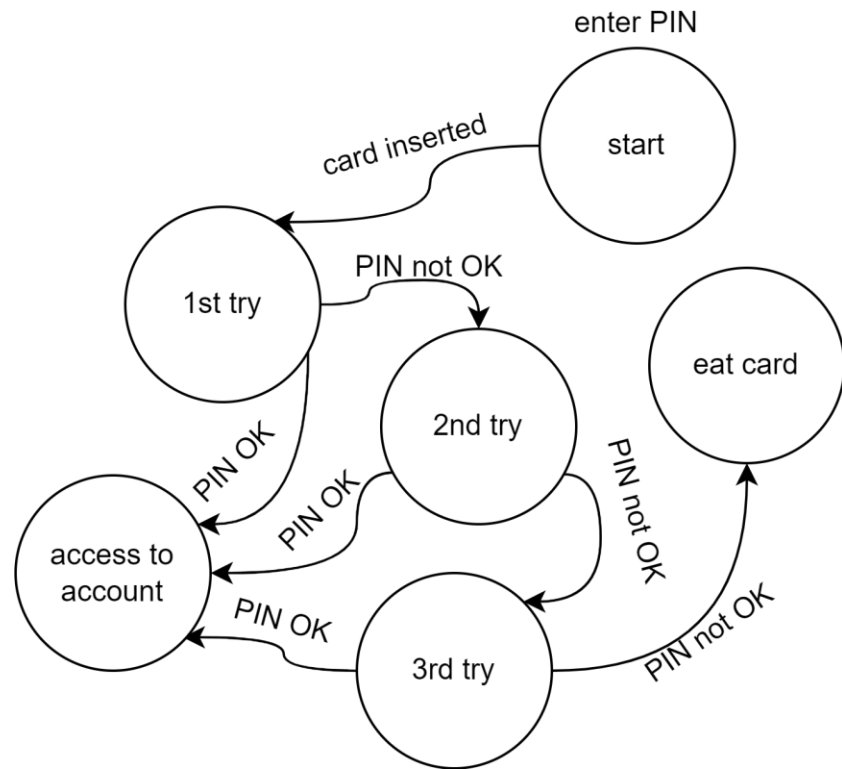
2. All Transitions Coverage

- **Requirement:** Exercise all valid transitions **AND** attempt to execute invalid transitions
- **Defect Prevention:** Helps avoid **Defect Masking** (where one defect prevents the detection of another)
- **Measurement:**
- **Evaluation:** Achieving full all transitions coverage guarantees both full all states coverage and full valid transitions coverage

State transition testing technique

Example: A function of an ATM machine

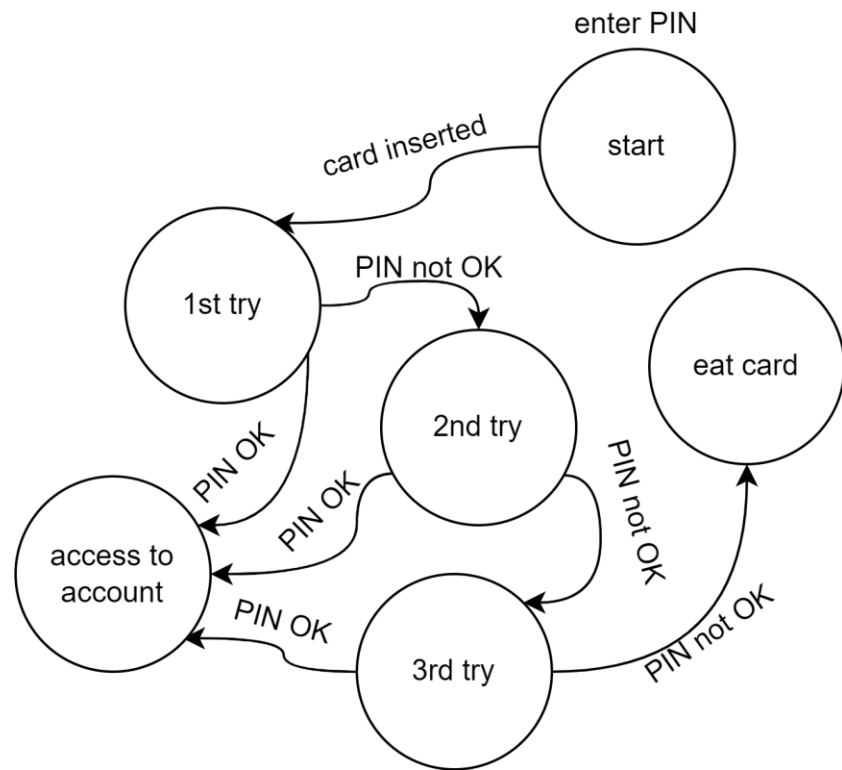
All states coverage:
There are 2 paths will
reach all states
coverage



State transition testing technique

Example: A function of an ATM machine

All transitions
coverage: There are 4
paths will reach all
transitions coverage



Pairwise testing technique

- **The Problem:** When there are many input parameters, testing all combinations (**Exhaustive Testing**) is impossible
- **The Solution (All-Pairs):** Uses combinatorics to ensure **every pair** of values is tested together at least once
- **Supporting Tools:** Classification Trees
- **Coverage Levels:**
 - 1-wise (single-wise)
 - 2-wise (Pairwise)
 - 3-wise...

Pairwise testing technique

Example:

	Class 1	Class 2	Class 3
Type	A	B	C
Size	L	M	N
Format	X	Y	Z

1-Wise Coverage		
Type	Size	Format
A	M	Y
B	L	Z
C	N	X

Pair Wise Coverage		
Type	Size	Format
A	L	X
A	M	Y
A	N	Z
B	M	Z
B	N	X
B	L	Y
C	N	Y
C	L	Z
C	M	X

Advantages of Black box testing

- **Broad Scope:** Enables the majority of testing classes, especially system performance like load and availability tests
- **Fewer Resources:** Significantly more resource-efficient than white box testing

Disadvantages of Black box testing

- **Masked Errors:** Coincidental errors might counteract each other to accidentally produce a correct output
- **No Coverage Control:** Cannot specify test cases to improve line coverage, leaving code unexecuted
- **No Code Quality Check:** Impossible to test internal coding quality or adherence to standards

Summary

- Definition and objectives
- Black box testing techniques
 - Equivalence partitioning technique
 - Boundary value analysis technique
 - Decision table technique
 - State transition testing technique
 - Pairwise testing technique